PERSONAL STUDY GUIDE / 73997

Lab 2 explanation

G M Robi Farhan Rojob
Machine Learning [CII5SE039CI], Group 01

Use this guide to understand the code well enough to draw the network, explain a line, and work through a small example without reading a prepared answer. The submission ZIP is separate; this PDF is for your preparation.

What you built

A program receives 64 pixel values from an 8 by 8 image and predicts a digit from 0 to 9. Two hidden layers transform those values. Training adjusts the weights using examples with known answers. Everything in the model is calculated with NumPy.

A useful order for studying

1. Start with the Lab 1 connection on page 2.
2. Follow data, shapes, forward propagation and loss on pages 3-6.
3. Work through backpropagation and gradient checking on pages 7-9.
4. Learn the training loop, results and timing on pages 10-12.
5. Practise code navigation and questions on pages 13-16.

The two Lab 1 sources

Your earlier Kaggle work was **AI Lab 1: Simple Artificial Neuron**. It covered weighted sums, Purelin, Logsig and manual parameter choices. The Machine Learning Lab 1 manual is a different document: it covers environment setup, baseline models and measurements. Its global rules also apply to Lab 2. This guide uses the first for concepts and the second for reproducibility and submission structure.

Before submitting

Review the three conclusions in report.md and express them in words you understand. The manual requires individual work and a commit in your own repository. After checking the results and practising the derivation, save your reviewed work in that repository and make a normal commit.

STEP 1 / CONNECT TO LAB 1

From one neuron to a network

In Lab 1, a neuron first calculated a weighted sum, then applied an activation function. That same operation is repeated many times in Lab 2.

```
n = x1*w1 + x2*w2 + b
a = activation(n)
```

Recall your first numerical example

The inputs were (1, 0), weights (1, -1), and bias 0.5. The sum is 1(1) + 0(-1) + 0.5 = 1.5. Purelin returns 1.5. Logsig returns 1/(1+exp(-1.5)) = 0.817574..., which rounds to 0.8176.

Recall the AND exercise

With weights (1, 1) and bias -1.5, only input (1,1) produces a positive sum. A sigmoid output of at least 0.5 is classified as 1. Scaling to (10,10) and bias -15 gives outputs closer to 0 and 1. You chose those values manually; no learning algorithm did it.

Recall the linear exercise

Weights (0,2) and bias 2 gave the rule a = 2*x2 + 2. Purelin could produce targets 2 and 4; sigmoid could not, because its mathematical output lies strictly between 0 and 1.

Lab 1	Lab 2
Two inputs	64 pixel inputs
One neuron	Two hidden layers and ten outputs
Manually selected weights	Weights updated by mini-batch SGD
Purelin or sigmoid	ReLU hidden layers; softmax output
Four tiny examples	1,797 labeled digit images

Say it aloud: A layer is a collection of neurons. Each neuron still computes a weighted sum plus a bias. The matrix multiplication calculates all of those sums for a whole batch at once.

STEP 2 / RUN_LAB.PY: PREPARE_DATA

Prepare the data

The dataset is built into scikit-learn. It contains 1,797 grayscale digit images. Each has 8 rows and 8 columns, so flattening gives 8 x 8 = 64 features. Labels are integers 0 through 9.

1. Divide every input by 16

The original intensity range is 0 to 16. Dividing by 16 gives a fixed range from 0 to 1: [0, 8, 16] becomes [0, 0.5, 1]. This puts inputs on a controlled scale. It is not standardization to zero mean and unit variance.

2. Encode the labels

```
digit 3 -> [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]
```

A one-hot vector has one correct class marked with 1. The other classes are 0. This lets cross-entropy select the log probability assigned to the correct digit. The number 3 is a category here, not a target whose numerical distance from 4 should be minimized.

3. Split before training

The stratified 70/30 split keeps approximately the same proportion of each digit in both sets. It gives **1,257 training samples and 540 test samples**. random_state=42 reproduces the same split. The model learns only from training examples; the test labels are used for final evaluation.

4. Keep the test set out of the training loop

The six settings are prescribed by the assignment. Each is trained for exactly 50 epochs, and the test score is calculated afterward. If you choose the winning setting from these test scores, that score is a comparison result, not an untouched final estimate after selection. A separate validation set would be needed for normal model selection.

Check yourself: Why is it acceptable to divide the test data by 16? Because 16 is a known scale from the dataset definition, not a statistic estimated by looking at test labels.

STEP 3 / NETWORK.PY: INITIALIZE

Draw the network and its shapes

```
64 inputs -> W -> ReLU -> W -> ReLU -> 10 -> softmax
W is either 16 or 64
```

Do not confuse **W**, the chosen hidden width, with **W1, W2 and W3**, which are matrices of learned weights. B is the number of examples in the current batch.

Quantity	Shape	At B=32, W=16
X	(B,64)	(32,64)
W1 / b1	(64,W) / (W,)	(64,16) / (16,)
Z1 and A1	(B,W)	(32,16)
W2 / b2	(W,W) / (W,)	(16,16) / (16,)
Z2 and A2	(B,W)	(32,16)
W3 / b3	(W,10) / (10,)	(16,10) / (10,)
Logits, P and Y	(B,10)	(32,10)

The rule is (a,b) @ (b,c) = (a,c). Therefore (32,64) @ (64,16) gives (32,16). Bias addition broadcasts a length-16 vector across the 32 rows. There is one bias per neuron, not one bias per example.

Count the parameters

```
64*W + W + W*W + W + W*10 + 10
= W*W + 76*W + 10
W=16: 1,482 parameters    W=64: 8,970 parameters
```

Initialization

Weights are random, mean 0 and standard deviation 0.01; biases are zero. Random weights break symmetry between hidden neurons. Setting every weight to zero would leave hidden neurons indistinguishable and, with this ReLU derivative, stop useful hidden-layer gradients.

Notation detail: N(0,0.01) sometimes denotes variance 0.01. The submitted report states explicitly that this experiment uses standard deviation 0.01. If your professor means variance, change the standard deviation to 0.1 and rerun every result.

STEP 4 / NETWORK.PY: FORWARD

Follow the forward pass

```
Z1 = X @ W1 + b1
A1 = maximum(0, Z1)
Z2 = A1 @ W2 + b2
A2 = maximum(0, Z2)
Z3 = A2 @ W3 + b3
P  = softmax(Z3)
```

Z means a value before activation. A means a hidden activation. Z3 contains raw scores called logits. P contains class probabilities. The cache stores intermediate values so backward propagation can reuse them.

ReLU

ReLU means max(0,z). It sends [-2, 0, 3] to [0, 0, 3]. Its derivative is 0 for negative values and 1 for positive values. At exactly 0 it is not differentiable; this code chooses 0.

Without nonlinear activations, several affine layers collapse into one affine transformation. ReLU makes it possible to learn more complicated relationships between pixels and digits.

Softmax: calculate a small example

For illustrative logits [2,1,0], subtract the maximum 2 to obtain [0,-1,-2]. Exponentials are approximately [1,0.3679,0.1353]. Dividing by their sum gives probabilities [0.6652,0.2447,0.0900]. The class with score 2 is predicted.

Subtracting the same constant from every logit cancels in the softmax ratio, so probabilities do not change. It avoids exp(large number) overflowing. For the real model there are ten logits, not three.

What must be true?

For a batch of B images, P must have shape (B,10). Values must be finite and nonnegative, and each row must sum to 1 within 1e-6. The source checks the shape and row sums before training.

Prediction: P.argmax(axis=1) chooses one digit per row. axis=1 means across class columns. A 0.5 threshold was useful for the binary AND exercise, but is not needed to choose among ten mutually exclusive digits.

STEP 5 / NETWORK.PY: CROSS_ENTROPY

Understand the loss

Accuracy counts correct choices. Cross-entropy also measures how much probability the model gives to the correct answer. That smooth information provides a useful training signal.

```
One example: loss = -log(probability of correct digit)
Batch loss: mean of the individual losses
```

True-class probability	Loss (natural logarithm)
0.9	0.105361
0.5	0.693147
0.1	2.302585
0.01	4.605170

A confidently wrong prediction receives a large penalty. One-hot encoding makes the expression -sum(Y*log(P)) keep only the correct-class term in each row.

Why is the initial loss about 2.3026?

Small random weights initially produce very similar logits. Ten nearly equal probabilities are close to 0.1 each. Then -log(0.1) = log(10) = 2.302585. Initial loss near this number is a sanity check, not proof that the gradients are correct.

Measured initial loss for width 16: **2.30258349**.

Measured initial loss for width 64: **2.30259491**.

Why calculate loss from logits?

A very small softmax probability can round to zero, making log(P) problematic. The code instead calculates log probabilities as shifted_logits minus log(sum(exp(shifted_logits))). This stable form is mathematically equivalent to softmax followed by the log.

Check yourself: Can accuracy stay unchanged while loss improves? Yes. If the predicted class stays correct but its probability rises from 0.6 to 0.9, the accuracy is unchanged and the loss decreases.

STEP 6 / CHAIN RULE

Backpropagation: the idea

A gradient tells you how a small change in a parameter changes the loss. Backpropagation calculates these derivatives efficiently, starting at the output and moving toward the input.

Start with one scalar chain

```
z = w*x + b
a = ReLU(z)
L = (a-y)^2 / 2
dL/dw = (a-y) * ReLU'(z) * x
```

For x=2, w=0.5, b=0 and y=0, z=1, a=1, and dL/dw=1*1*2=2. A positive gradient means that increasing w increases the loss locally. Gradient descent subtracts a positive amount to reduce w. This example explains the chain rule; the submitted classifier uses cross-entropy, not this squared loss.

The convenient softmax result

Combining softmax with cross-entropy simplifies the output derivative to P-Y for one sample. For a mean over B samples, divide once by B:

```
D3 = (P - Y) / B
```

Suppose the true class is the second of three illustrative classes. P=[0.2,0.3,0.5] and Y=[0,1,0], so P-Y=[0.2,-0.7,0.5]. The negative middle entry encourages increasing the true-class logit. Positive entries encourage reducing the competing logits.

What the chain rule does at each layer

1. Calculate the gradient of that layer's weights.
2. Sum over the batch for its bias gradient.
3. Multiply by the weight transpose to send the gradient to the previous activations.
4. Apply the previous ReLU mask so inactive neurons pass zero gradient.

Do not mix up the roles: Backpropagation computes gradients. The optimizer uses them to change parameters. The backward function itself does not update the weights.

Backpropagation: every equation

```
D3  = (P - Y) / B
dW3 = A2.T @ D3
db3 = D3.sum(axis=0)

D2  = (D3 @ W3.T) * (Z2 > 0)
dW2 = A1.T @ D2
db2 = D2.sum(axis=0)

D1  = (D2 @ W2.T) * (Z1 > 0)
dW1 = X.T @ D1
db1 = D1.sum(axis=0)
```

Why is there a transpose?

At width 16, A2 has shape (B,16), so A2.T has shape (16,B). Multiplying (16,B) by D3 of shape (B,10) gives (16,10), exactly the shape of W3. The multiplication accumulates each hidden activation's contribution to each output gradient over the batch.

Why sum biases over axis 0?

The same bias is added to every example. Each row contributes a derivative, and all contributions must be added. Summing the B rows of D3 leaves a length-10 vector. axis=1 would sum classes instead and give the wrong shape.

Why multiply by Z > 0?

A negative ReLU input produced a constant output zero, so its local derivative is zero. Multiplying by the Boolean mask applies this derivative element by element. In NumPy, @ is matrix multiplication and * is elementwise multiplication; interchanging them changes the calculation.

Two common mistakes

Double averaging: D3 already includes division by B. Dividing again in dW2 or dW1 incorrectly weakens earlier gradients.
Updating too early: Replacing W3 before computing D2 mixes two different parameter states. Compute every gradient first, then update all arrays.

Paper practice: Cover the equations and reconstruct them from the output backward. State the shape of every product. That is more reliable than memorizing the code without understanding it.

Check the gradients numerically

A decreasing loss can still hide a mistake. Numerical checking estimates a slope without using the backpropagation formula, then compares the two answers.

```
g_num = [L(w + epsilon) - L(w - epsilon)] / (2*epsilon)
epsilon = 0.00001
relative_error = abs(g_analytic - g_num) /
                 max(1e-12, abs(g_analytic) + abs(g_num))
```

The exact sequence

1. Use the initial network and the first eight training examples.
2. Calculate analytical gradients once.
3. With seed 42, choose ten different weight coordinates.
4. Save a selected weight, increase it by 1e-5 and calculate loss.
5. Decrease it by 1e-5 relative to its original value and calculate loss.
6. Restore the original weight, calculate the centered slope, and compare.
7. Keep every sampled coordinate, including failures. Require at least 9/10 relative errors below 1e-6 before training.

Width 16: 10/10 passed. Largest relative error: 2.048e-14.

Width 64: 10/10 passed. Largest relative error: 2.053e-14.

Why use Decimal for the reference calculation?

At initialization, two losses near 2.3026 can differ only in very late digits. Subtracting them in ordinary float64 can lose accuracy. The independent reference uses Python's built-in Decimal with 50 digits. The model and its analytical derivatives still use NumPy float64. This is numerical differentiation, not automatic differentiation.

ReLU has a corner at zero. If a perturbation changes an activation's sign, a centered slope may not match the derivative at the original point. The CSV records such crossings. The denominator floor prevents division by zero for zero gradients; zero-gradient matches are weaker evidence than nonzero matches.

Train the model

Gradient descent changes each parameter in the opposite direction of its gradient. The learning rate controls how large that step is.

```
parameter = parameter - learning_rate * gradient
```

For weight 0.4, gradient 0.2 and learning rate 0.1, the next weight is 0.38. If the gradient is -0.2, the next weight is 0.42. The sign comes from the loss derivative, not from whether a weight is currently positive or negative.

Follow one epoch

1. Shuffle the indices of the 1,257 training examples.
2. Select up to 32 indices.
3. Run forward propagation for that mini-batch.
4. Calculate gradients with backward propagation.
5. Update each weight and bias.
6. Continue through the remaining batches.
7. Recalculate full-training-set loss and accuracy and write the epoch record.

There are 39 full batches of 32 and one final batch of 9, giving **40 updates per epoch**. Over 50 epochs, each experiment makes **2,000 updates**. The last batch must divide gradients by 9, not 32; the code uses its actual length.

Why six experiments?

Two widths multiplied by three learning rates gives six runs: (16,0.001), (16,0.01), (16,0.1), (64,0.001), (64,0.01), and (64,0.1). Each starts again from fresh weights. Continuing from the previous model would make the comparison unfair.

Why shuffle and seed?

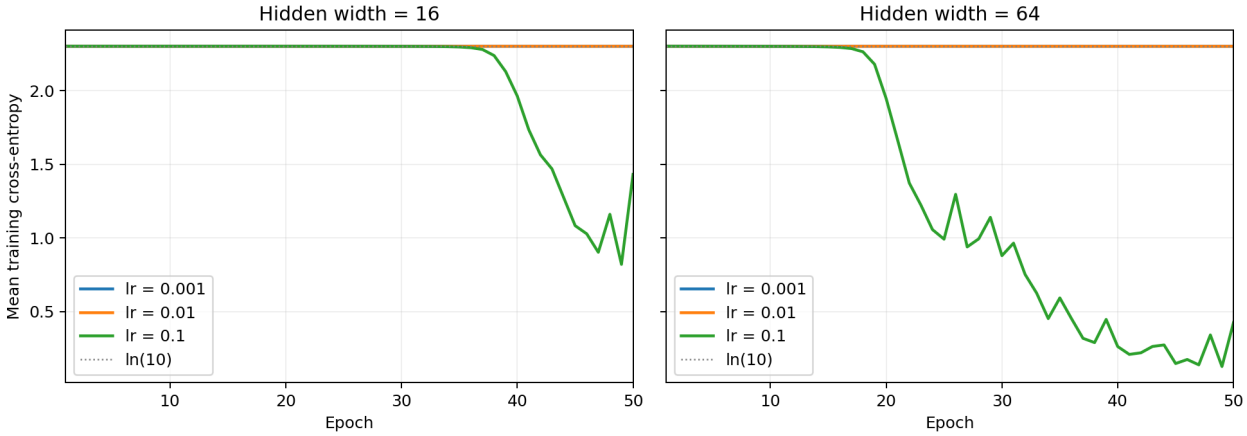
Shuffling avoids repeatedly presenting examples in one fixed order. Seeding makes the experiment reproducible. The same initialization and shuffle sequence are used across rates at each width, so differences are easier to attribute to the learning rate. Seeds do not make hardware timing identical or guarantee the same floating-point result on every platform.

Prediction after training: only forward propagation is needed. A new digit image does not require its true label, a backward pass, or a weight update.

STEP 10 / RESULTS/SUMMARY.CSV

Read your actual results

Width	Rate	Train loss	Test accuracy
16	0.001	2.30255	10.19%
16	0.01	2.30247	10.19%
16	0.1	1.43238	47.22%
64	0.001	2.30252	10.19%
64	0.01	2.30226	10.19%
64	0.1	0.42659	86.11%



Read the axes before interpreting a curve

The horizontal axis is epoch 1 to 50. The vertical axis is mean training cross-entropy. Lower is better. The dotted reference is ln(10), the loss of a uniform ten-class prediction. A flat curve near that level indicates little learning under this schedule.

Your highest measured test score is **86.11%**, using width **64** and learning rate **0.1**. Say 'in these six runs', rather than claiming it is the best possible digit classifier.

Small initial weights can produce small activations and gradients through both hidden layers. A very small learning rate may therefore make little progress in 50 epochs. This explains the observed low-rate behavior as a plausible mechanism; it is not a controlled proof that initialization alone caused it.

Loss does not have to decrease at every epoch in mini-batch SGD. Different batches and a relatively large step can cause oscillations. A larger network has more capacity, but more parameters alone do not guarantee better test accuracy.

Explain computational timing

The timer measures one forward call and one backward call separately. The batch size is fixed at 32. Each operation has one warm-up, then 100 measured calls. The median is the middle value after sorting (for 100 values, the average of positions 50 and 51).

Width	Rate	Forward us	Backward us	Ratio
16	0.001	17.80	18.95	1.06
16	0.01	17.90	19.30	1.08
16	0.1	18.05	19.05	1.06
64	0.001	38.90	51.45	1.32
64	0.01	38.50	51.10	1.33
64	0.1	39.05	51.30	1.31

One microsecond (us) is one millionth of a second. A ratio above 1 means backward took longer. Values here are measured on the current machine, not promised performance on Kaggle or the laboratory computer.

What is inside and outside the timers?

Forward includes the three linear layers, two ReLUs, softmax and saving a cache. Backward includes weight gradients, bias gradients and ReLU masks. Its cache is prepared before timing. Neither timer includes parameter updates, loss evaluation, printing or plotting.

Why does backward usually take more work?

```
Forward matrix work:  B*(64*W + W*W + 10*W)
Backward matrix work: B*(64*W + 2*W*W + 20*W)
```

Forward has three matrix products. Backward has five: three weight-gradient products and two products to propagate gradients to earlier hidden layers. It does not calculate the input gradient, because pixels are not learned parameters. Matrix costs alone do not determine exact wall time; elementwise operations and memory allocation also matter.

Why median? It is less affected by occasional slow calls from scheduling or background activity. **Why one BLAS thread?** These matrices are small; extra thread coordination can dominate their actual arithmetic.

STEP 12 / SUBMISSION NAVIGATION

Know where the work is saved

File	What you should be able to explain
lab02/report.md	Goal, method, results and exactly three conclusions; no code listings.
lab02/src/network.py	Initialization, forward pass, loss, backward pass and evaluation.
lab02/src/gradient_check.py	Independent central-difference check using Decimal.
lab02/src/run_lab.py	Split, train all settings, plot and measure timings.
lab02/src/print_versions.py	Actual Python and package versions, including absent course packages.
lab02/results/gradient_check.csv	Each sampled coordinate, both slopes, relative error, kink flag and pass.
lab02/results/epoch_log.csv	300 rows: six experiments times 50 epochs.
lab02/results/timing_raw.csv	600 pairs: six configurations times 100 calls.
lab02/results/model_*.npz	The six learned parameter sets.
requirements.txt	The exact course-wide list, preserved as supplied.

Run from the repository root

```
python lab02/src/print_versions.py
python lab02/src/run_lab.py
python -m unittest discover -s lab02/src -p test_network.py
```

Use Python 3.11.8 in an isolated environment. The README explains the environment conflict and the working Lab 2 requirements file. Running the experiment overwrites measurement files. The report records the supplied run; if you rerun and obtain different results, update its tables and conclusions to match.

PRACTICE / ANSWER BEFORE READING

Random questions: foundations

Why exactly 64 inputs?

An image has 8 x 8 pixels; flattening produces one feature for each pixel.

Why ten outputs?

There are ten mutually exclusive classes, digits 0 through 9.

What does a bias do?

It shifts a neuron's preactivation independently of the input values. There is one bias per neuron.

Why not use sigmoid for the output as in Lab 1?

Ten independent sigmoid values need not sum to one. Softmax makes the ten classes compete as one probability distribution.

Why not use Purelin in every layer?

Composing affine transformations is still one affine transformation. Hidden nonlinearities add expressive power.

What is the difference between a logit and a probability?

A logit is an unconstrained score. Softmax converts all logits together into normalized probabilities.

What is an epoch, a batch and an iteration?

An epoch visits the complete training set. A mini-batch is a subset used for one update. Here an iteration means one such update.

What is the role of scikit-learn in this model?

Only dataset loading and the stratified split. It does not implement the network or train it.

What is the role of PyTorch?

Only the seed calls required by the global manual. It performs no model computation or automatic differentiation.

PRACTICE / EXPLAIN THE REASON

Random questions: training

Why is D3 equal to (P-Y)/B?

The chain rule combines the softmax Jacobian with cross-entropy to give P-Y; dividing by B corresponds to the mean loss.

Why is the bias gradient a sum?

The shared bias contributes to every sample's preactivation, so the sample contributions accumulate.

What if you forget the ReLU mask?

You propagate gradients through inactive neurons as if their activation were linear, so the derivative no longer matches the forward computation.

Can a gradient check pass when the model learns poorly?

Yes. Correct derivatives do not guarantee useful initialization, learning rate, capacity or training duration.

Why not pick a much smaller finite-difference step?

Truncation error becomes smaller, but floating-point cancellation can get worse. The assignment fixes the step at 1e-5.

Why not change the seed until every check passes?

That would hide failed coordinates. This check uses a fixed seed and records every selected weight.

What would happen with learning rate zero?

Parameters would not change. Nonzero gradients alone do not update a model.

Why are some gradients zero?

An input can be zero or a ReLU path can be inactive. A zero result is not automatically a bug, but many inactive paths may hinder learning.

How many updates are made per run?

ceil(1257/32) = 40 per epoch; 40 x 50 = 2,000 updates.

A short oral rehearsal

A 30-second explanation

The program classifies 8 by 8 digit images. It normalizes the 64 pixels, passes them through two ReLU hidden layers and uses softmax for ten class probabilities. Cross-entropy measures error, backpropagation gives the gradients, and mini-batch SGD updates the parameters. The gradients are checked numerically before training. Six settings are compared using loss, accuracy and separate forward/backward timings.

Five small tasks to do without the code

1. Draw the 64-W-W-10 architecture and label every weight shape.
2. Calculate ReLU([-2,0,3]) and state the derivative mask.
3. Calculate the loss when the correct digit has probability 0.1.
4. Update weight 0.4 using gradient -0.2 and learning rate 0.1.
5. Explain why a final batch of nine uses division by nine.

Answers: (64,W), (W,W), (W,10); [0,0,3] with mask [0,0,1]; 2.302585; 0.42; because the implemented loss is the mean over the actual batch.

If the teacher points to a line

First identify the input shape, then say what operation happens, then state the output shape and its purpose. For example: 'A2.T is W by B; D3 is B by 10. Their product is W by 10, the gradient of the output-layer weights.' This method works even if you forget a variable name.

If asked about a limitation

Be specific: the variance convention is ambiguous; the full package list has a dependency conflict; sampled checks do not prove every derivative; timings are machine-dependent; and selecting settings using test scores is not a fully independent evaluation. Explain how each limitation affects the interpretation of the results.

Finish in your own repository

Extract the ZIP into your existing course repository, review and rerun the work, and revise the three conclusions in your own words. Recreate the architecture and chain-rule derivation on paper as requested. Make your own normal commit after review. The personal study PDF is not part of the submission ZIP.

Sources used for this guide: the supplied Machine Learning Lab 1 global rules and Lab 2 manual, the completed AI Lab 1 neuron notebook from your earlier chat, and the actual source code and measurement files in this submission. The assigned textbook sections remain part of your pre-lab reading.